

universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

DevSecOps

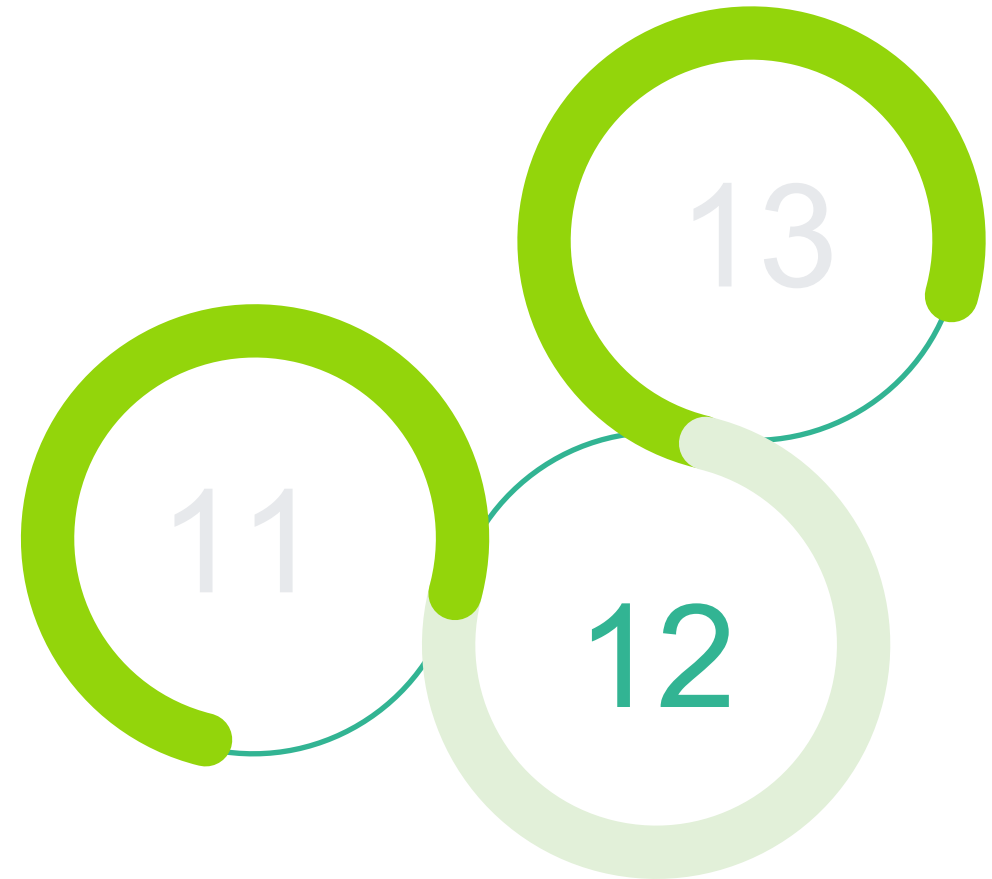
Software Engineering

2025/2026



Summary

- DevOps
- DevSecOps
 - Security applied to all the phases
- Security shift

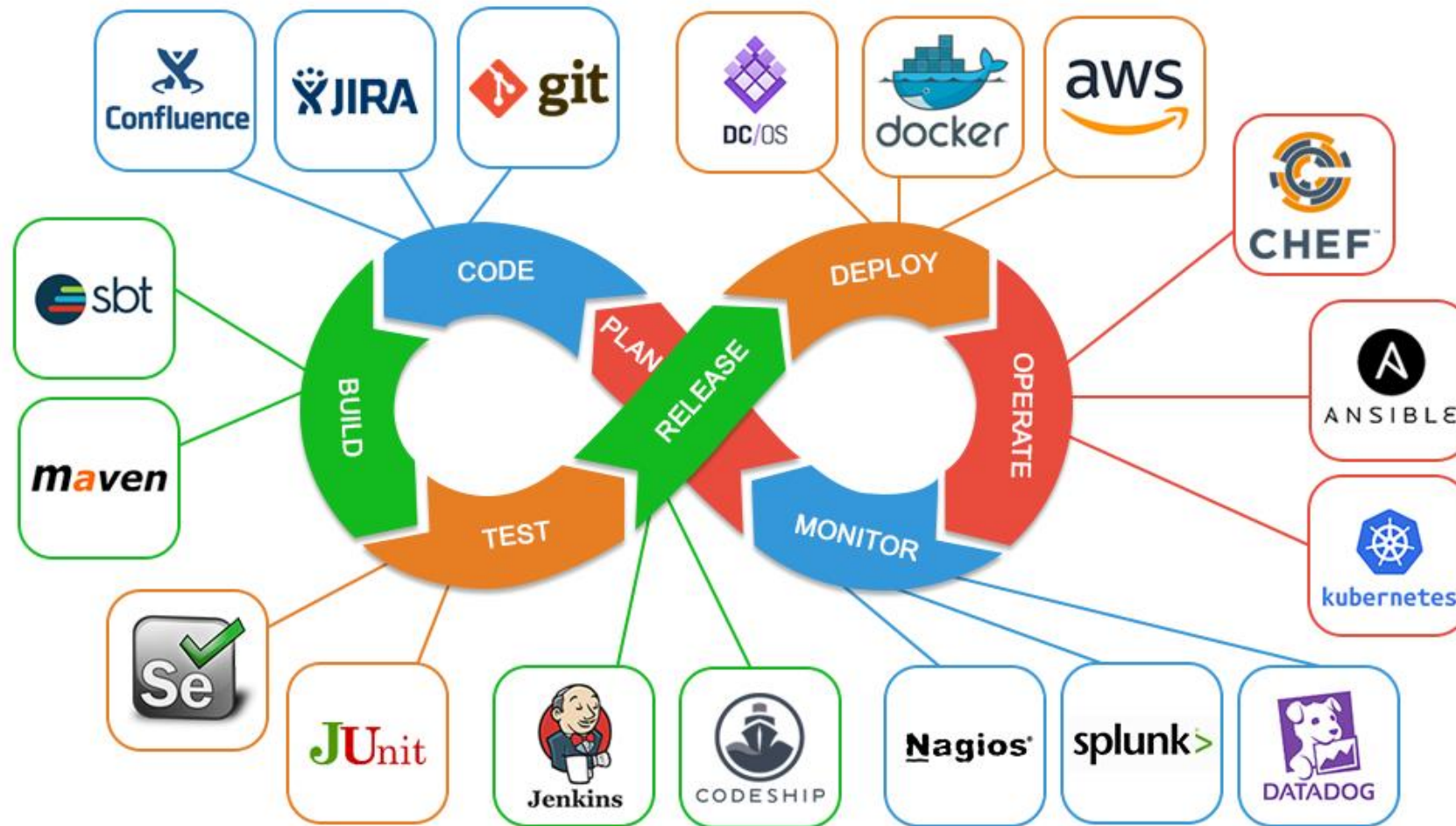


Dev vs Ops

- The **development** team kicks things off
 - by “throwing” a software release “over the wall” to Operations.

- **Operations** picks up the release artifacts and begins preparing for their deployment.
 - They also hand edit configuration files to reflect the production environment
 - which is **significantly different than the Development environments**.

DevOps



Why DevOps wasn't enough

- DevOps **increased deployment frequency** (from months to minutes).
- Traditional **security checks are too slow** for CI/CD.
 - Manual pentesting, audits, ...
- Rapid code changes introduce vulnerabilities faster than security teams can catch them manually.
- High-performing DevOps teams deploy 208x more frequently.
 - Security must match this pace.

Why DevOps wasn't enough

Vulnerabilities growing

- Essential due to the rapid growth of vulnerabilities
 - (~315,000 in NVD) → <https://nvd.nist.gov/general/nvd-dashboard>

➤ Example:

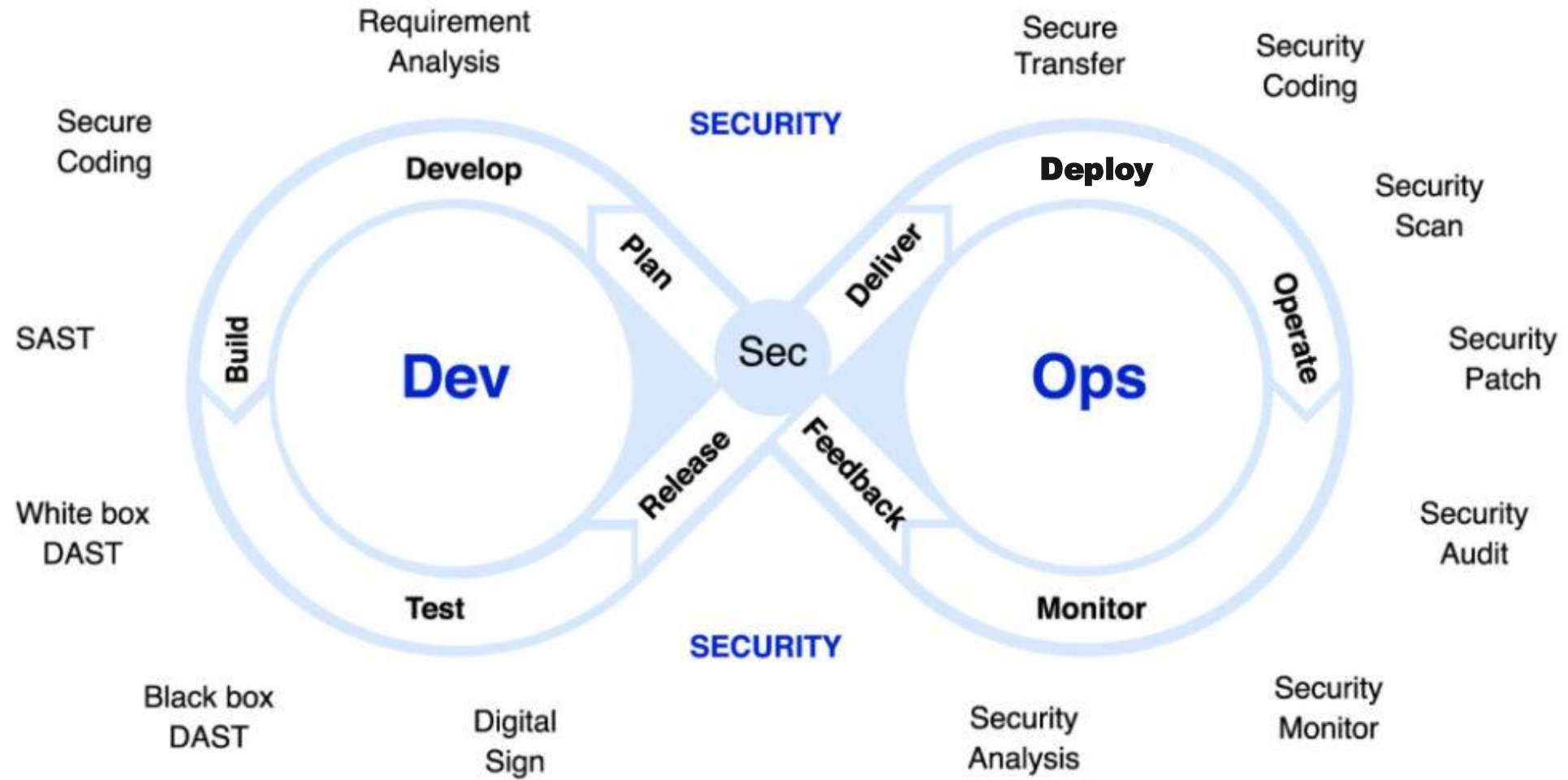
- 38% of attacks in 2023 started via a vulnerability

Time Period	New CVEs Received by NVD	New CVEs Analyzed by NVD	Modified CVEs Received by NVD	Modified CVEs Re-analyzed by NVD
Today	191	85	0	129
This Week	191	85	0	129
This Month	3745	3464	0	496
Last Month	4536	3797	0	319
This Year	40499	34221	0	3138

Dev Ops meets Security → DevSecOps

- Integrating security practices into the DevOps process from the start.
- Motto: "Security is everyone's responsibility."
- From "Security as a Gate" to "Security as a Guardrail."
- The goal is delivering secure software without slowing down development velocity.

DevSecOps



Planning phase

Threat modeling (The "What If")

- Analyzing the architecture to identify potential threats *before* development begins.
 - Identifying Trust Boundaries: Where does data cross from public to private?

	Threat	Definition	Security Control (The Fix)
S	Spoofing	Pretending to be someone else.	Authentication (MFA, Strong Sessions)
T	Tampering	Modifying data in transit or on disk.	Integrity (Digital Signatures, Checksums)
R	Repudiation	Claiming "I didn't do that."	Non-Repudiation (Audit Logs)
I	Info Disclosure	Exposing private data.	Confidentiality (Encryption, ACLs)
D	DoS	Crashing the system.	Availability (Rate Limiting, CDNs)
E	Elevation of Priv	Gaining admin access.	Authorization (Least Privilege)

Planning phase

"Abuse Cases" vs. Use Cases

- A user story often follows the following “equation”:

As a <type of user>, I want <some feature> so that <reason>

- An abuse case can follow a different “equation”:

As a <type of attacker>, I want <exploit of that feature> so that <negative consequence>

Component	Standard User Story (The Builder)	Abuse Case / Evil Story (The Breaker)
Who?	"As a Customer ..."	"As an Attacker ..."
What?	"...I want to reset my password ..."	"...I want to abuse the reset function ..."
Why?	"...so that I can recover my account ."	"...so that I can enumerate valid emails ."

Planning phase

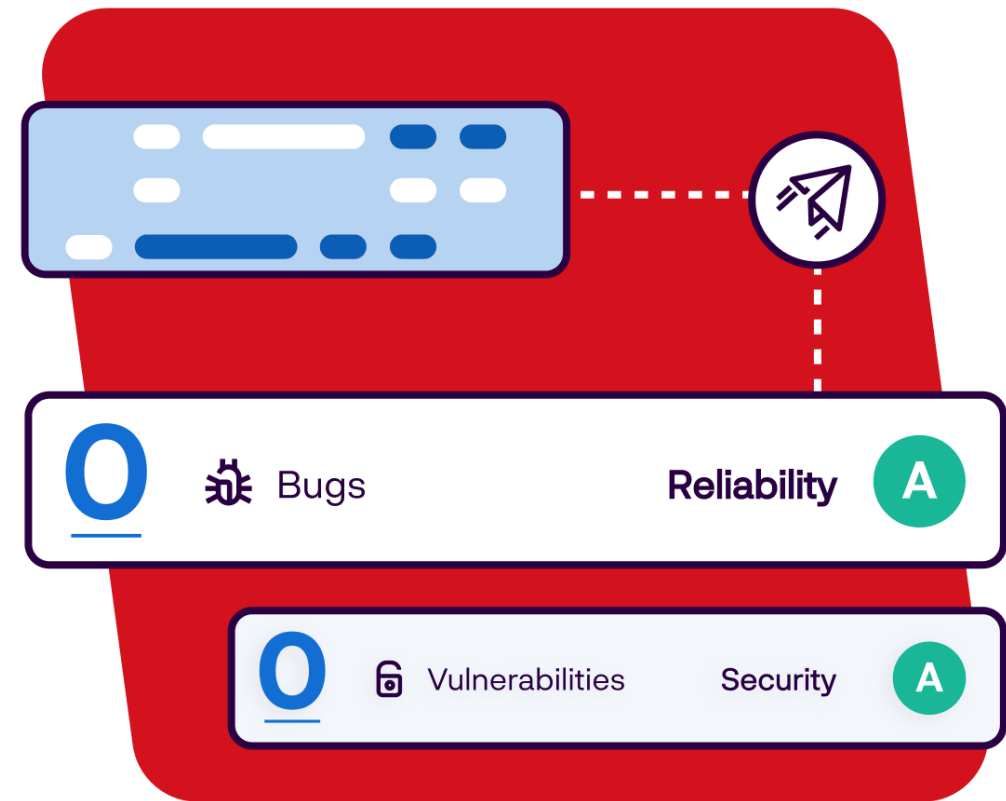
Security requirements as code

- Security constraints are entered into the backlog (e.g., Jira/Trello) alongside functional requirements.
 - Example: "Auth token expiration shall be set to 15 minutes."
- Definition of Done (DoD)
 - A feature is not "finished" until it passes specific security criteria (e.g., code reviewed, scans passed).

Coding phase

IDE Security Plugins

- Tools (e.g., SonarLint, Snyk) don't just say "Error."
 - They explain why it is insecure and provide a "Quick Fix" button.
- Provides instant remediation advice (how to fix it) without leaving the editor.
- Developers learn security patterns while they code.
 - *Acts like Grammarly or Microsoft Word Spellcheck, but for security flaws.*



Coding phase

Software Composition Analysis (SCA) - Local

- 80-90% of a modern application is open-source code (libraries, frameworks).
 - You are responsible for the security of every library you import.
- Checking open-source libraries/dependencies for known vulnerabilities (CVEs) before importing them.
- “Don't inherit other people's security holes.”



Coding phase

Pre-commit hooks

- A script that runs automatically on the developer's laptop the moment they try to commit code.
 - If the script fails, the commit is rejected.

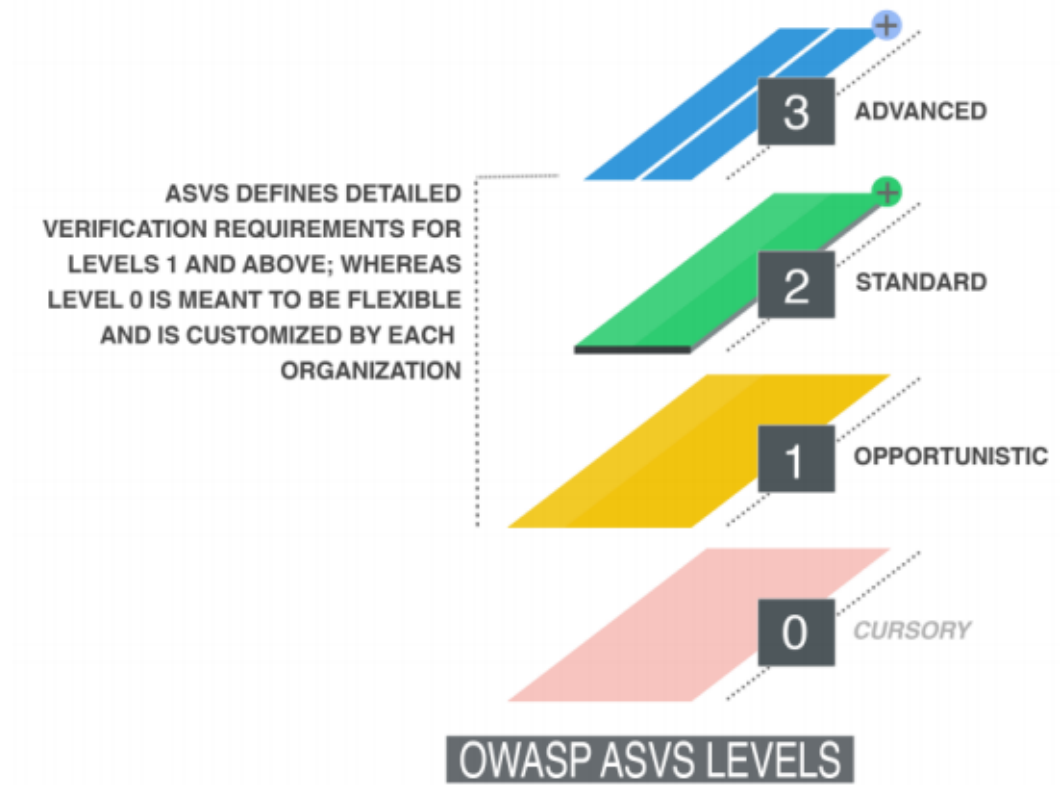
- The #1 Use Case: Secret Scanning
 - **Problem:** Accidental commits of AWS keys, DB passwords, or API tokens.
 - **Solution:** The hook regex-scans the changes. If it looks like a key, the commit is blocked.
 - **Result:** Secrets never leave the laptop (preventing "Secret Sprawl").

- Linting: Enforces coding standards automatically.

Coding phase

Security-conscious peer review

- Code reviews check for logic flaws and security implications.
- The Security Checklist for Reviewers:
 - Does this code handle untrusted input?
 - Are we logging sensitive data (PII) to plain text logs?
 - Is the user authorized to perform this specific action?



Building phase

SAST (Static Application Security Testing)

➤ White-Box Testing:

- Scans the uncompiled source code to find vulnerabilities (e.g., SQL Injection, XSS, Buffer Overflows).

➤ Inter-file analysis

- IDE Plugins check the file you are editing.
- Build-Phase SAST traces data flow across the entire project.
- Example: Identifying that a variable created in File A is used insecurely 10 layers deep in File Z.

➤ The audit trail

- Unlike local checks, this scan generates a permanent record of the code's health for compliance and tracking.

Building phase

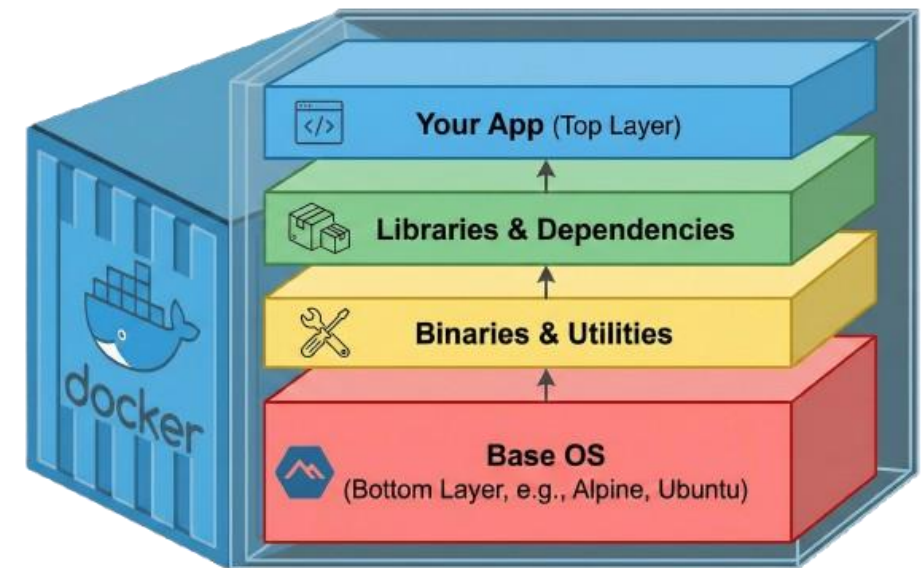
Breaking the build

- Rule: If a vulnerability > **High** severity is detected, the pipeline stops immediately.
- Prevents insecure code from ever being packaged or deployed.
- Problems: If the build breaks too often (False Positives), developers will bypass security.
 - Only break the build for High Confidence + High Severity issues.

Building phase

Container security & base image scanning

- A Container = Your App (Top Layer) + Libraries + Binaries + Base OS (Bottom Layer).
- You can have secure Java code running on a vulnerable version of Alpine Linux.
- What we scan:
 - **The Dockerfile:** Checks for bad practices
 - e.g., **running as root**, exposing port 22.
 - **The Base Image:** Checks against CVE databases
 - for known OS vulnerabilities.



Building phase

Artifact signing

➤ Problem:

- A hacker compromises the build server and injects malware into the binary after the tests pass but before it is deployed (e.g., SolarWinds attack).

➤ Cryptographically signing the compiled binary or container image.

- Ensures that what is built is exactly what gets deployed (preventing tampering).



Testing phase

DAST (Dynamic Application Security Testing)

➤ Black-Box Testing:

- The tool interacts with the running application exactly like a hacker would (without seeing the source code).

➤ SAST sees the code; DAST sees the reality.

- Finds: Environment issues SAST misses, such as:
- Misconfigured HTTP Headers (CORS, HSTS).
- Authentication/Session bypasses.
- Reflected Cross-Site Scripting (XSS).

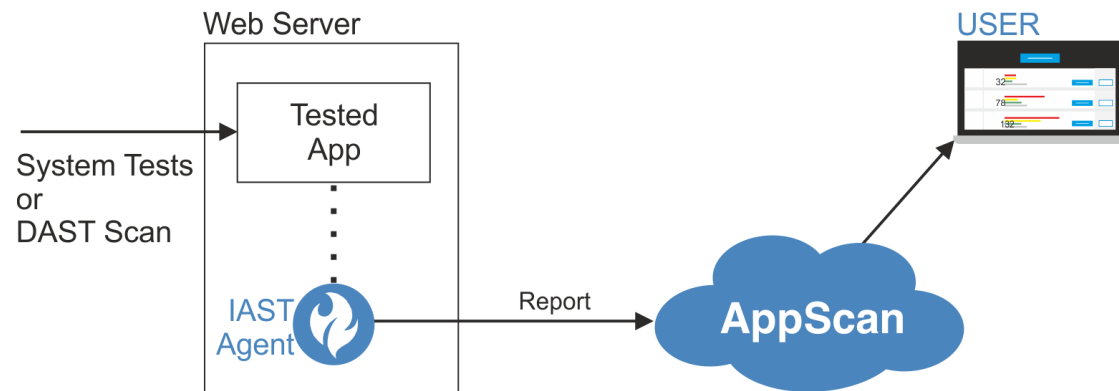
➤ Slower than SAST; requires a deployed, running environment.

Testing phase

IAST (Interactive Application Security Testing)

- An agent sits *inside* the application (like a debugger) while DAST runs.
 - SAST says: "This might be a bug."
 - DAST says: "Something weird happened here."
 - IAST says: "I watched the attack enter the function and trigger the vulnerability. This is real."

- Verifies if an attack actually triggered a vulnerability in the code, reducing false positives significantly.



Testing phase

Fuzz testing (Fuzzing)

- Bombarding APIs and input fields with random, malformed, or unexpected data ("garbage inputs").
 - Triggering edge cases that developers never thought to write tests for.
- System crashes, memory leaks, or unhandled exceptions
 - Which often lead to exploits.
- Why it matters:
 - Humans write logical tests (e.g., "Enter age: 25").
 - Fuzzers write chaotic tests (e.g., "Enter age: -999999" or "Enter age: 🦀").

Release phase

Infrastructure as Code (IaC) Scanning

- In the cloud, infrastructure is just text files (Terraform, K8s manifests).
 - **Statistic:** Gartner estimates that 99% of cloud failures will be the customer's fault (misconfiguration).

- Static analysis for Infra
 - Scanners (e.g., Checkov, tfsec) parse your Terraform/CloudFormation code.
 - **What they find:**
 - "S3 Bucket is set to Public."
 - "Database is missing encryption at rest."
 - "Security Group allows traffic from 0.0.0.0/0 (Anywhere)."

- Fixing a text file takes 1 minute.
 - Fixing a breach caused by an open bucket takes months.

Release phase

Secrets management (no more hardcoding)

- Never, ever commit API keys, passwords, or certificates to Git.
 - Even private repos leak.
- The modern solution (the vault pattern)
 - Dynamic Injection: The application does not know the database password.
- The Workflow:
 - App starts up and authenticates with the Vault (using IAM role).
 - Vault generates a temporary credential (valid for 1 hour).
 - App uses the credential.
 - Credential expires automatically.
- If a hacker steals the password, it's useless an hour later.

Release phase

Policy as Code

- Decoupling decision logic from application logic.
 - Policy as Code (PaC) is the Building Inspector. They don't build anything; they just walk around with a clipboard (Rules) and say, "That window is too small—it's against code. Tear it down."
- The Gatekeeper: runs right before deployment.
- Examples of Rules:
 - Compliance: "Deployments are only allowed to 'EU-West' (GDPR)."
 - Cost: "No generic developer can launch an instance larger than t2.large."
 - Security: "Container images must come from our internal Trusted Registry."
- Compliance is no longer a manual audit; it is a binary pass/fail check in the pipeline.

Release phase

Immutable infrastructure

- Do not patch live servers.
 - Replace them.
- If a vulnerability is found in the OS, you deploy a fresh, patched image and destroy the old one.
 - This prevents "Configuration Drift."
- If a hacker compromises a server and installs a backdoor, that backdoor is destroyed the next time you deploy.
 - Attackers cannot maintain a foothold in an immutable environment.

Deploy phase

RASP (Runtime Application Self-Protection)

- Security agents embedded inside the running application.
- Can intercept calls to the database or filesystem in real-time.
- Action: If it sees a SQL injection attack, it blocks the request without crashing the app.

Deploy phase

The feedback loop

- Security incidents in production must automatically become tickets in the "Plan" phase backlog.
- Real-world attack data informs the next Threat Modeling session.
- Safe Deployment Strategies
 - Canary Deployments: Rolling out the update to only 1% of users first to check for security stability.
 - Blue/Green Deployment: Maintaining two environments to allow instant rollback if a vulnerability is exploited.

Operate phase

Full-Stack Observability (Logs, Metrics, Traces)

- Moving beyond "Is the server up?" to "Is the server behaving normally?"
 - Some classes ago, we talked about monitoring
- What about malicious actors?
- Centralized Logging (SIEM)
 - Aggregating logs from the App, OS, Cloud, and Network into one searchable place (e.g., Splunk, ELK).

Operate phase

Automated Incident Response (SOAR)

- Security Orchestration, Automation, and Response.
- The "glue" that connects your security tools (SIEM, Firewall, ChatOps) to execute actions automatically.
- Scenario: A suspicious IP begins scanning Port 22 (SSH).
 - Manual Way: Analyst sees alert → Investigates IP → Logs into Firewall → Adds Rule
 - (Time: > 20 mins)
 - SOAR Way: Playbook triggers → Firewall blocks IP instantly without human intervention
 - (Time: 5 seconds)

Operate phase

Active security testing

- Red Teaming: Hiring ethical hackers to attack the production system periodically.
- Bug Bounty Programs: Paying external researchers to find vulnerabilities you missed.
- The Feedback Loop (Post-Mortems)
 - Every incident generates a "Blameless Post-Mortem."
 - The findings become new user stories in the "Plan" phase. (e.g., "We were hacked because of X, so we will add a check for X in the CI pipeline.")

Monitor phase

SIEM + UEBA

➤ Centralized Logging & SIEM

- Collecting logs from the entire stack (Application, OS, Database, Network) into one place.
- Single Source of Truth: You cannot investigate an attack if you have to log into 50 different servers manually.

➤ UEBA (User and Entity Behavior Analytics)

- Moving beyond static rules (e.g., "Alert on 5 failed logins").
- Using Machine Learning to establish a "baseline of normal" and flagging deviations (e.g., "User Bob is accessing sensitive HR files at 2 AM on a Sunday").

Monitor phase

Continuous monitoring

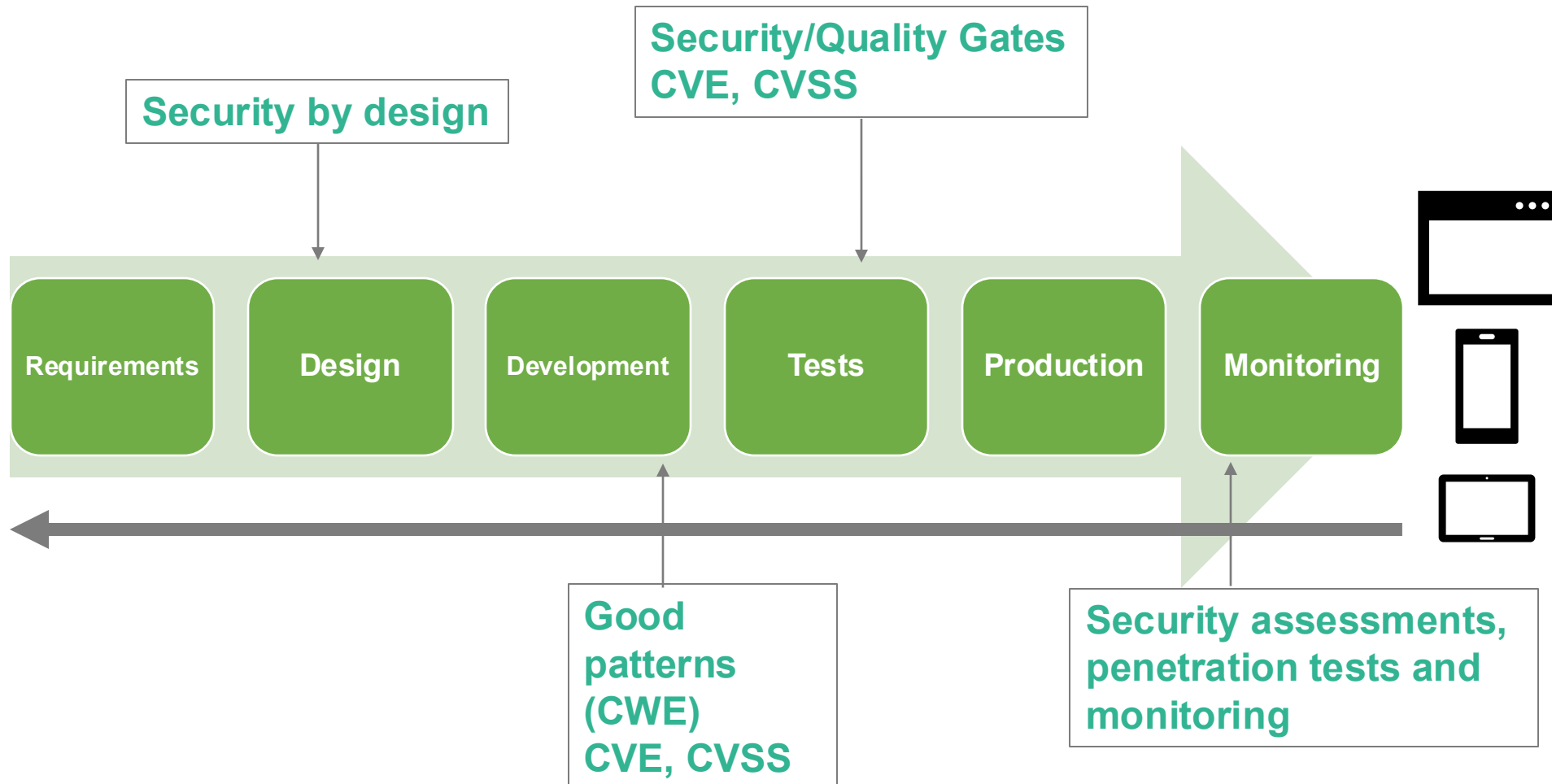
➤ Continuous Compliance

- Automated scanners that check the live environment 24/7 against standards (GDPR, HIPAA, PCI-DSS).
- Alerts immediately if a configuration "drifts" out of compliance (e.g., an encrypted drive was decrypted).

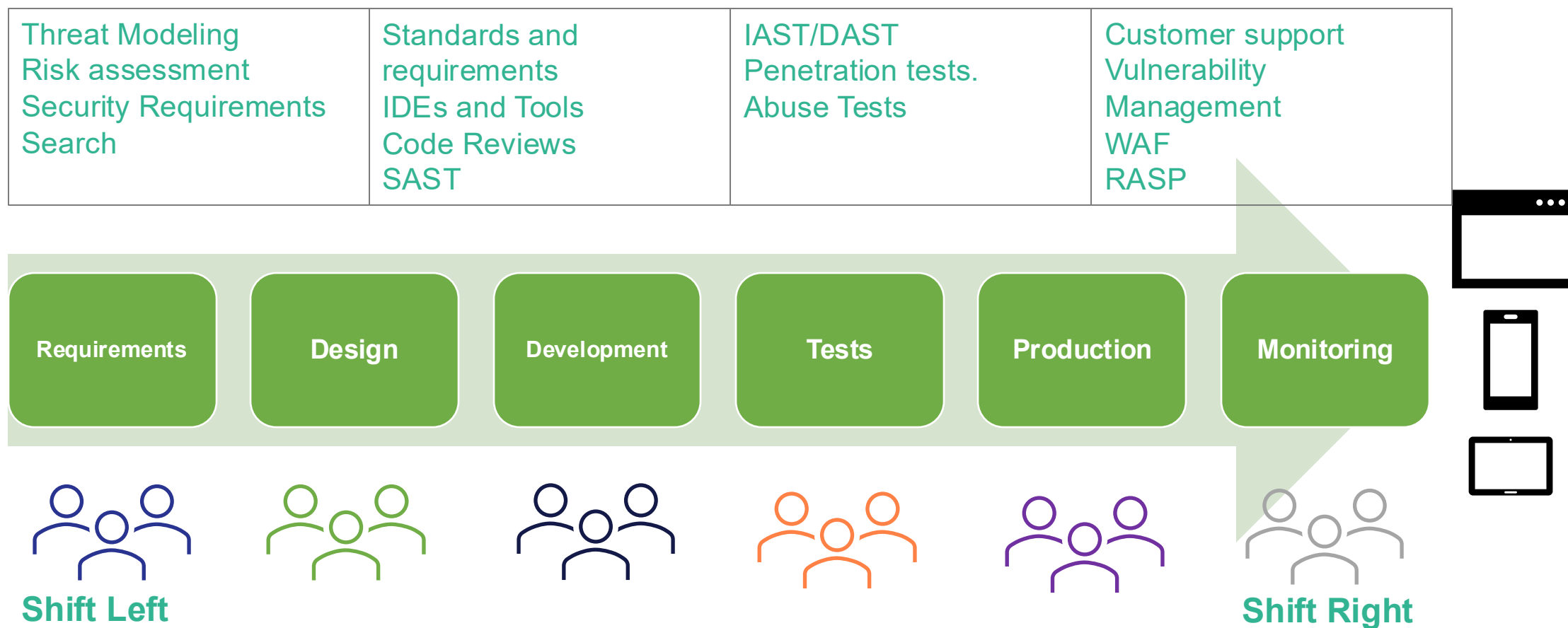
➤ Key Security Metrics (KPIs)

- MTTD (Mean Time To Detect): How long does a threat live before we see it?
- MTTR (Mean Time To Recover): How fast can we patch the hole?

Security shift



Security shift



**If you want to learn more
about this subject...**



**...you should try Security in
Software Engineering**